Cross compiling FreeTDS to iPhone

Since this question is unanswered and I spent the majority of a semester figuring it I thought I would post how to Cross compiling FreeTDS 0.91 to iPhone ARMv6, ARMv7 architecture. This was done using Xcode 4.2 and iOS 5 SDK.

The reason this question is asked it because you are developing an app for an iOS device that requires connecting to an Mircosoft SQL Sever, which requires using the Tabular Data Stream (TDS) protocol as it is Microsoft proprietary.

I will also mention that you need some level of technical skill to even attempt this. This is a very condensed version of what took me nearly two months to figure out (I left all the thinks you shouldn't do).

Other documentation relating to this:

Basic How To on using FreeTDS http://www.freetds.org/userguide/samplecode.htm

Microsoft's TDS API documentation http://msdn.microsoft.com/en-us/library/aa936985(v=sql.80)

See my answer below.

iphone | ios | xcode4.2 | cross-compiling | freetds

edited Nov 16 '12 at 2:57

asked Jul 11 '12 at 2:49
Tristan
156 2 9

1 Tristan, I wish I could upvote several times. – magma Jul 20 '12 at 20:10

1 This is great! But I ended up using iSQL SDK by mobilefoo. – Hackmodford Nov 4 '12 at 5:31

Tristan, you should *answer your own question* below, so you can then accept your own question. – Amigable Clark Kant Nov 15 '12 at 13:46

@AmigableClarkKant That would make a lot more sense wouldn't it. – Tristan Nov 16 '12 at 2:49

add comment

start a bounty

2 Answers

For those like me that will spend hours finding the documentation for these standard configure flags (for running ./configure make make install)

```
./configure --build is used for specifing the architecture you want to com
./configure --host is used to specify the ark of the machine doing the com
./configure --target seems to be an alias
```

Now then to solving the problem.

1) Get the latest version of the FreeTDS http://www.freetds.org/

2) The next step is to make your own bash shell files that correctly run the FreeTDS ./configure. You will need two as the simulator is i386/i686 architecture and an apple device (iPhone, iPod, etc.) is ARM architecture. Also, your compiler files/version within the iPhone development directories may be different, just find what makes logical sense and has similar naming convention. The mac host architecture is supplied with the command uname -p.

Here is my example for building for use on the simulator (i386) build_for_simulator_i386.sh:

```
#!/bin/sh

#unset some shell variables
unset CC
unset CFLAGS
unset CPP

export buildPath=`pwd`

# make i386 (Simulator) target
export CC=/Developer/Platforms/iPhoneSimulator.platform/Developer/usr/bin/i686-ap

export CFLAGS="-isysroot /Developer/Platforms/iPhoneSimulator.platform/Developer/

# if you want Windows Authentication (NTLM) support you must use at least tds ver
# the default is 5
./configure --build=i386 --host=i386 --target=i386 --with-tdsver=7.1
```

Example for configuring for ARM compilation (build_for_device_armv7.sh):

```
#!/bin/sh

# unset some shell variables
unset CC
unset CFLAGS
unset CPP

export buildPath=`pwd`

# make arm target
export CC=/Developer/Platforms/iPhoneOS.platform/Developer/usr/bin/arm-apple-darw

export CFLAGS="-isysroot /Developer/Platforms/iPhoneOS.platform/Developer/SDKs/iP

export CPP=/usr/bin/cpp


./configure --build=arm-apple-darwin10 --host=x86_64-apple-darwin11.3.0 --target=
```

3) Next cd to the root freetds directory that resulted from unzipping the freetds download, mine was freetds_0.91

4) Run one of your scripts. You can only compile for **ONE** architecture at a time

```
sh build_for_(desiered build)
       this runs ./configure for you with the correct options
       (tds version 7 required for NTLM authentication)
```

5) Once the configure process completes you have to hack the configuration file. Open freetds_0.91/include/config.h then on line 172 change **#define HAVE_ICONV 1** to **#define HAVE_ICONV 0**

6) If you previously ran ./configure, make, make install then run these commands. Especially if your switching architectures as you will get errors running make without doing this

```
sudo make clean
sudo make uninstall
```

7) Perform the compilation using make

```
make all
sudo make install
```

The make procedure does through some error on purpose, but if you see errors within six or seven lines of shell prompt, once it returns, you have problems and need to fix them before proceeding. Lets just say lots of things can go wrong at this point.

8) After installing the binary complied file that is the culmination of all the little .o files that freetds makes is **/usr/local/lib/libsybdb.a** Trust me you don't want to pull a .o file for just the library you want. Copy /usr/local/lib/libsybdb.a to the appropriate folder in your project. What I did was have two separate folders, one per architecture, named "compiled_freetds-0.91_simulator_i386" and "compiled_freetds-

0.91_device_armv7."

9) Since you want to make you life easy and have xcode figure out which compiled file to use follow this subset of steps to perform the dynamic linking.

```
a) Select you project settings on the left had side of xcode
(the blue think with the name of your project on it)

b) Select the Target (usual the same name as your app)

c) Navigate to **build settings**, scroll down to **linking > other linker flags*

d) On the left side of Other Linker Flags a mouse over will reveal an expander,
expanding will reveal Debug and Release rows.

e) Add the appriate architectures by selecting the plus on the right side of
either Debug or Release.  When the new row appears select the architecture,
double click the first editable field from the right to open  an entry box
that you can then drag the appropriate  complied file into it to be dynamically
linked.  You must do this for both files and when done correctly the file
under ARMv7 will be used when building for the device and the one for Any iOS
Simulator SDK will be used when running on the simulator.
**Note:** You may also need to add the -all_load flag to resolve linking issues.
```

10) The final step which seems to avoid problem of dynamic linking error involving libsybdb.5.dylib when running code on device is to make uninstall. Also, when running on the device you will also get lots of warnings, in increments of 36, about CPU_SUBTYPE_ARM_ALL being deprecated, that is normal, but annoying.

```
    sudo make uninstall
```

I hope this helps.

answered Nov 16 '12 at 2:47



Tristan

156 2 9

add comment

I used the above bash files but since XCode 4.5 the Developer Tools are inside the app bundle. So I modified the scripts to run with my MacOS Lion and the current XCode Version "4.5.2 (4G2008a)"

build_for_simulator_i386.sh:

```
#!/bin/sh

# unset some shell variables
unset CC
unset CFLAGS
unset CPP

# make i386 (Simulator) target
export CC=/Applications/Xcode.app/Contents/Developer/Platforms/iPhoneSimulator.plat
export CFLAGS="-isysroot /Applications/Xcode.app/Contents/Developer/Platforms/iPhor
export CPP=/usr/bin/cpp

./configure -build=i686-apple-darwin11 --host=i686-apple-darwin11 --target=i686-app
```

build_for_device_armv7.sh:

```
#!/bin/sh

# unset some shell variables
unset CC
unset CFLAGS
unset CPP

# make arm target
export CC=/Applications/Xcode.app/Contents/Developer/Platforms/iPhoneOS.platform/D
export CFLAGS="-isysroot /Applications/Xcode.app/Contents/Developer/Platforms/iPho
export CPP=/usr/bin/cpp

./configure --build=arm-apple-darwin10 --host=x86_64-apple-darwin11 --target=armv7
```

A nice addon is to use lipinfo to merge the two static libraries into one by

```
lipo compiled_freetds-0.91_device_armv7/libsybdb.a compiled_freetds-0.91_simulator
```

and just adding this to the project settings.

Wanted to share it, since the above scripts saved me a lot of time.

edited Jan 20 at 13:37

answered Jan 19 at 19:23



saskathex

11 2

add comment

Not the answer you're looking for? Browse other questions tagged iphone | ios | xcode4.2 | cross-compiling | freetds **or ask your own question.**